Hyperparameters:

```yaml
# Dataset
xsum:
  dataset_name: EdinburghNLP/xsum
  text_column: document
  summary_column: summary
  max_source_length: 768
  max_target_length: 64
  pad_to_max_length: false
  val_max_target_length: 64
  num_beams: 4

# Baseline: BART inference
bart:
  model_name: facebook/bart-large-xsum
  batch_size: 8
```

```yaml
# Finetuning models
finetune:
  t5_base:
    model_name: google-t5/t5-base
    lr: 3.0e-5
    batch_size: 16
    gradient_accumulation_steps: 2
    num_train_epochs: 50
    fp16: false
    early_stopping_patience: 2
    early_stopping_threshold: 0.0
    lora:
      enabled: false
      r: 16
      alpha: 32
      dropout: 0.05
      target_modules: [q, k, v, o]
  llama-1b:
    model_name: meta-llama/Llama-3.2-1B-Instruct
    lr: 2.0e-5
    batch_size: 8
    gradient_accumulation_steps: 8
    num_train_epochs: 50
    fp16: true
    early_stopping_patience: 2
    early_stopping_threshold: 0.0
    lora:
      enabled: true
      r: 16
      alpha: 32
      dropout: 0.05
      target_modules: [q_proj, k_proj, v_proj, o_proj]
```

```yaml
# MoE-from-scratch
moe:
  d_model: 512
  nhead: 8
  num_encoder_layers: 4
  num_decoder_layers: 4
  dim_feedforward: 2048
  dropout: 0.1
  vocab_size: 32128
  max_position_embeddings: 1024
  num_experts: 8
  k: 2  # top-k tokens per router
  router: topk   # options: topk, hash
  load_balancer_weight: 0.01
  use_lora_experts: false
  lora:
    r: 16
    alpha: 32
    dropout: 0.05

# Generation
generation:
  num_beams: 4
  max_new_tokens: 64
  length_penalty: 1.0
  no_repeat_ngram_size: 3
```

Note that to perform Bonus 3, use_lora_experts was switched to true and the router was alternated between topk and hash for necessary experiments. For bonus 1 (ie. MoE without load balancer), load balancer weight was changed to 0.

Compute

For SummaC evaluation, a separate venv was created on Ada (4x NVIDIA GeForce RTX 2080; 11 GB VRAM each)

For the remaining code, it was run on external compute (NVIDIA A100 80GB PCIe; 80 GB VRAM)

Results:

BART:

```
rouge1: 0.5654855185540558
rouge2: 0.4015418293664637
rouge3: 0.3217565328359306
rouge4: 0.2705458538444897
rougeL: 0.5065243574692653
bleu: 32.54392561152994
bertscore_p: 0.6033354997634888
bertscore_r: 0.5467334985733032
bertscore_f1: 0.5747990608215332
compression_ratio: 0.07001643632199213
extractiveness: 0.7949499269077017
summac_conv: 0.41522224979942374
```

Fine-tuned T5 (google-t5/t5-base):

```
rouge1: 0.47956176861872624
rouge2: 0.250317541139682
rouge3: 0.14899583998683058
rouge4: 0.09496082632436849
rougeL: 0.39712059124447496
bleu: 15.958971776297426
bertscore_p: 0.5649589896202087
bertscore_r: 0.4726148843765259
bertscore_f1: 0.5184263586997986
compression_ratio: 0.07381434427545365
extractiveness: 0.6886148226959906
summac_conv: 0.3267064509036623
```

Fine-tuned Llama (meta-llama/Llama-3.2-1B-Instruct):

```
rouge1: 0.38310557390480543
rouge2: 0.16365583433663955
rouge3: 0.08385275994298089
rouge4: 0.04791830626281117
rougeL: 0.30750749817774875
bleu: 10.239866370939664
bertscore_p: 0.4290787875652313
bertscore_r: 0.3629603683948517
bertscore_f1: 0.39586490392684937
compression_ratio: 0.08467392647072143
extractiveness: 0.49462101880268017
summac_conv: 0.29696353488333105
```

MoE Hash (with load balancer):

```
rouge1: 0.2068734827637331
rouge2: 0.04761073543489647
rouge3: 0.014023063440904086
rouge4: 0.005092296917628559
rougeL: 0.16101604492124783
bleu: 2.1016221698605273
bertscore_p: -0.08880097419023514
bertscore_r: 0.07842936366796494
bertscore_f1: -0.005736777558922768
compression_ratio: 0.15075869785747367
extractiveness: 0.30134426873138637
summac_conv: 0.3417951205819845
```

MoE topk (with load balancer):

```
rouge1: 0.22507016871587976
rouge2: 0.05665089760950869
rouge3: 0.0184404872425963
rouge4: 0.007313734540730926
rougeL: 0.17333221054887124
bleu: 2.5528519865664268
bertscore_p: -0.056617897003889084
bertscore_r: 0.10086773335933685
bertscore_f1: 0.02174448035657406
compression_ratio: 0.14424198767007834
extractiveness: 0.3247469861074334
summac_conv: 0.28645681098103526
```

MoE GQA:

```
rouge1: 0.0477363318695902 54
rouge2: 0.0031687127806592544
rouge3: 0.0002862449963230895
rouge4: 2.7691830145162216e-05
rougeL: 0.043338987567894 32
bleu: 0.0405724687985568 74
bertscore_p: -0.6774793267250061
bertscore_r: -0.1666315495967865
bertscore_f1: -0.436180382966995 24
compression_ratio: 0.140605481678627
extractiveness: 0.1779895846730315
summac_conv: 0.39542784830927846
```

MoE topk (without load balancer):

```
rouge1: 0.2228567018818911
rouge2: 0.05577039307183745
rouge3: 0.018311072222654174
rouge4: 0.007310474945602863
rougeL: 0.17035431443702279
bleu: 2.5883092493117155
bertscore_p: -0.009715046733617783
bertscore_r: 0.11551311612129211
bertscore_f1: 0.05291936174035072
compression_ratio: 0.1395534174021563
extractiveness: 0.3295582384386529
summac_conv: 0.3036006838530302
```

MoE hash (without load balancer):

```
rouge1: 0.21692790874873868
rouge2: 0.05208956064266045
rouge3: 0.01599809510178504
rouge4: 0.005918082454935071
rougeL: 0.16710820882366953
bleu: 2.260648505141227
bertscore_p: -0.0900980681180954
bertscore_r: 0.08582326024770737
bertscore_f1: -0.0028542429208755493
compression_ratio: 0.14823492061290836
extractiveness: 0.32379807672959077
summac_conv: 0.3184913890808821
```

MoE topk (with LoRA):

```
rouge1: 0.185845564860876
rouge2: 0.03633913306193835
rouge3: 0.008982710756529382
rouge4: 0.00280393973583176
rougeL: 0.14602774878776087
bleu: 1.4947989173649188
bertscore_p: -0.1601303070783615
bertscore_r: 0.04203622788190842
bertscore_f1: -0.060250263661146164
compression_ratio: 0.15401118445945647
extractiveness: 0.25665595956708287
summac_conv: 0.3084040079265833
```

MoE hash (with LoRA):

```
rouge1: 0.1710988349918665
rouge2: 0.03137042088491383
rouge3: 0.007249210682353317
rouge4: 0.0022512995342963034
rougeL: 0.13676111441931635
bleu: 1.2114923121193626
bertscore_p: -0.24396346509456635
bertscore_r: 0.006742571014910936
bertscore_f1: -0.1215290054678917
compression_ratio: 0.1452027210910678
extractiveness: 0.28276498788793547
summac_conv: 0.31069648154079915
```

Models:

Fine-tuned Llama (meta-llama/Llama-3.2-1B-Instruct):
https://huggingface.co/ishro/llama-exsum

Fine-tuned T5 (google-t5/t5-base):
https://huggingface.co/ishro/t5-base-exsem

MoE Hash (with load balancer):
https://huggingface.co/ishro/moe-hash-xsum

MoE topk (with load balancer):
https://huggingface.co/ishro/moe-topk-xsum

MoE GQA:
https://huggingface.co/ishro/moe-topk-xsum-gqa

MoE topk (without load balancer):
https://huggingface.co/ishro/moe-topk-xsum-noload

MoE hash (without load balancer):
https://huggingface.co/ishro/moe-hash-xsum-noload

MoE topk (with LoRA):
https://huggingface.co/ishro/moe-topk-xsum-lora

MoE hash (with LoRA):
https://huggingface.co/ishro/moe-hash-xsum-lora

Human Evaluation Observations

Baseline BART:
In terms of context relevance, the BART examples managed to capture all the main points from the 3 examples I looked at. With respect to coherence, there was a clear smooth flow with a clear, cohesive structure. In terms of fluency, the grammar flowed very well and there were properly constructed sentences throughout the samples. Lastly, minor details are sometimes omitted from the source though this is expected given the task but for the most part, the factual accuracy is amazing. Overall, the baseline is as strong as the automated metrics show.

T5-base:
In terms of the context relevance, the T5 model is able to cover the key topics in a comprehensive manner matching the reference very well. Moreover, in terms of coherence, like the BART baseline, these texts have a near-perfect semantic flow and provide a good, strong narrative throughout. In terms of the grammatical quality, it is also perfect. Lastly, it is very strong in its factual accuracy towards the source material. There is not that much degradation in performance between the BART baseline and T5, though T5 was a little stronger in my opinion.

Llama:
In terms of context relevance, the Llama model underperforms compared to BART and T5 because it adds some interpretive details; this is often an issue with LLMs and other causal models. In terms of coherence, there is some awkward phrasing especially when compared with the previous models; however, overall, a generally-understandable flow is still maintained. In terms of grammar, it still maintains decent outputs though it can be improved upon. Finally, for factual accuracy there are a few major factual issues related to interpretation (for example, in one example, it identified “teacher” instead of "asbestos worker").

MoE hash:
In terms of contextual relevance, this model veers off topic quite a bit and struggles to identify the main points of the passages. In terms of the coherence, the output is very incoherent in multiple places, lacking logical connections or a unified message. Grammatically speaking, there are lots of incomplete sentences with poor structure. Lastly, with respect to factual accuracy, there are multiple hallucinations. This is the first model where the outputs are overall just significantly disappointing showing the advantage of using off-the-shelf standard models.

MoE hash with LoRA:
In terms of context relevance, the outputs are almost wholly unrelated to the source text. In terms of coherence, there is lots of runaway generation and lack of clear semantic flow. The grammar is, likewise, very broken with random periods and no clear structure present. Lastly, the very few details which seem to be conveyed seem to be either fabricated or not expounded upon correctly (mostly the former). This is one of the worst models

MoE hash with no load balancer:
This model is slightly better than the hashing with LoRA. Starting with context relevance, it does slightly touch upon the topic of the source text but the crucial points and details are missed. With regard to the coherence, once again, this model's output lacks a coherent structure like we saw with the original 3 models. In terms of grammar, once there are poorly structured constructions and sentence fragmentations throughout. In terms of the factual accuracy, while there are interpretable facts (unlike the previous model), the outputs are generally indecipherable and are not related to the source text.

MoE with top k:

In timers of contextual relevance, this model once again misses key points and includes a bunch of tangential information which is not expected in an extreme summarization task. The coherence is also lacking with very few smooth transitions between ideas. The grammar is riddled with issues though there is some evidence of basic sentence structure which does at least make the output readable. Lastly, the text is somewhat close to the facts but I would not describe it as completely accurate information.

MoE top k with LoRA:
Like the other LoRA models, this one again has very little contextual relevance in the sense that there is very little connection to the source material. The output is also not very coherent, consisting of various disjoint phrases instead of actual sentences. And once again, like the previous LoRA model, it has poor grammar with broken sentences. Lastly, it completely invents things like details and names which are not present in the source.

MoE top k with no load balancer:
A similar pattern to the hash MoE is observed where the output of the no load balanced output is better than LoRA top k and better (though in the case of top k close to) the results obtained from the top k with the load balancer. In terms of contextual relevance, some of the topics are partially identified but key details remain missing. In terms of the coherence, their structure remains fragmented with poor constructions. The grammar is also incredibly weak although you can see the model trying to attempt to form grammatical constructions. There are also lots of factual inaccuracies present including wrong professions, locations, and names.

MoE with top k + GQA:
Runaway generation means this output is practically unusable.

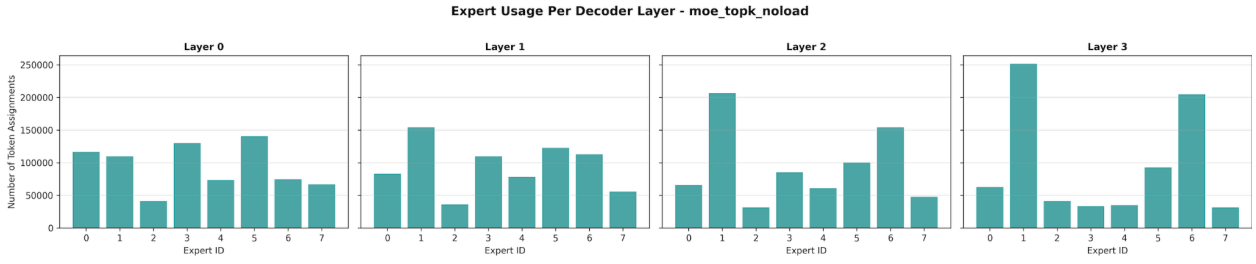
Bonus 1 Requirements

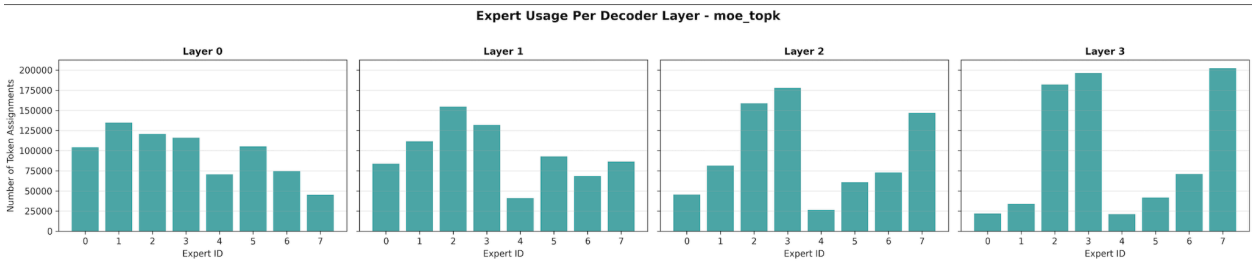
Metric	MoE Top-k (No LB)	MoE Top-k (With LB)
ROUGE-1	0.2229	**0.2251**
ROUGE-2	0.0558	**0.0567**
ROUGE-L	0.1704	**0.1733**
BERTScore-F1	**0.0529**	0.0217
BLEU	**2.5883**	2.5529
SummaC Conv	**0.3036**	0.2865

Metric	MoE Hash (No LB)	MoE Hash (With LB)
ROUGE-1	**0.2169**	0.2069
ROUGE-2	**0.0521**	0.0476
ROUGE-L	**0.1671**	0.161
BERTScore-F1	**-0.0029**	-0.0057
BLEU	**2.2606**	2.1016
SummaC Conv	0.3185	**0.3418**

My results have indicated that using the load balancer is bad for the quality of the summaries generated. This is highly-evident for the hash routing strategy as can be seen above as in every metric except SummaC, the model with no load balancer outperforms the model with load balancer. Even in top-k routing, while the model with load balancer does narrowly outperform in ROUGE, there are much more substantial differences in BERTSCore and SummaC pushing the top-k with no load balancer above.

This might be because my load balancer weight was not a good fit for the model (ie. it was 0.01) and perhaps with a different weight, I could have attained more optimal results. The goal of the load balancer is to prevent certain experts from collapsing but in this case, it could be suppressing good results.





Bonus 2 Requirements

```
rouge1: 0.21507016871587976
rouge2: 0.05565089760950869
rouge3: 0.0174404872425963
rouge4: 0.006313734540730926
rougeL: 0.16333221054887124
bleu: 1.2028519865664268
bertscore_p: -0.066617897003889084
bertscore_r: 0.0908677335933685
bertscore_f1: 0.01174448035657406
compression_ratio: 0.13424198767007834
extractiveness: 0.3147469861074334
summac_conv: 0.27645681098103526
```

The actual results didn't change much, but were slightly worse (probably due to the lack of efficiencies inherent in the library-built code). However, the inference time was significantly lower, around 5-6x lower.

Bonus 3 Requirements

Challenges:

1) One of the challenges I faced was what parameters to use for LoRA. LoRA comes with its own parameters, namely dropout, alpha (learning rate), and the rank ®. Ultimately I ended up using standard parameters I found online (https://huggingface.co/learn/llm-course/en/chapter11/4).
2) At a few points, the code failed to load due to errors I had made in properly initializing the matrices (specifically due to tensor mismatch). To pinpoint the issue, I needed to add debug print statements.

Simple Running instructions

1. Run BART Baseline
python scripts/baseline_bart.py --output runs/baseline_bart

2. Fine-tune T5
python scripts/finetune_t5.py --output runs/t5_base

3. Train MoE Model
python scripts/train_moe.py
  --output runs/moe_topk
  --router topk
  --epochs 10
  --batch_size 8

4. Evaluate MoE Model
python scripts/infer_moe.py
  --checkpoint runs/moe_topk/best_model
  --output runs/moe_eval

PLEASE NOTE: DETAILED RUNNING INSTRUCTIONS ARE IN THE GITHUB REPO